Capítulo 2

Etapas en la resolución de problemas utilizando una computadora. Expresiones: aritméticas, relacionales y lógicas. Funciones predefinidas.

1. Etapas en la resolución de problemas utilizando una computadora

Las personas aprenden lenguajes de programación principalmente para utilizar la computadora como herramienta para la resolución de problemas. En este sentido, pueden identificarse dos etapas en el proceso de resolución de problemas asistida por computadora:

1. Resolución del problema en sí
2. Implementación (realización) de la solución en la computadora.

El resultado de la primera etapa es el diseño de un algoritmo para resolver un problema.
El algoritmo se expresa en un lenguaje de programación que la computadora pueda entender (por ejemplo Pascal, C, entre tantos otros). Este algoritmo, expresado en cualquier lenguaje de programación de computadoras, se denomina programa.
El diseño de programas es una tarea compleja y un proceso creativo. No hay un conjunto completo de reglas para indicar como escribir programas. Sin embargo, se debe considerar el uso de lineamientos o conductas para la obtención de una solución computable (o sea que la solución se exprese mediante un programa de computadora), a saber:

- Análisis del problema
- Diseño del algoritmo
- Codificación
- Compilación y ejecución
- Verificación y depuración
- Documentación

Las dos primeras etapas conducen a un diseño detallado escrito en forma de algoritmo. Durante la codificación se implementa el algoritmo en un código escrito en un lenguaje de programación (código fuente), que refleja las ideas desarrolladas en las fases de análisis y diseño.

La compilación tiene como resultado el código de máquina, que es la traducción del código fuente, mediante el empleo de intérpretes o compiladores. A continuación el programa compilado es alimentado en la computadora para que ésta lo ejecute (indicando el punto de partida y los datos de entrada).

La verificación y la depuración son necesarias para encontrar errores y eliminarlos. Como recomendación es importante que mientras más tiempo se gaste en el análisis y el diseño, menos tiempo se invertirá en la verificación y depuración. Además, las etapas de codificación, compilación, ejecución, verificación y depuración se llevan a cabo en forma iterativa, lo que significa que al momento de verificar y depurar el programa, es necesario volver a codificar partes de él, compilar y ejecutar.

Además, se debe realizar la documentación del programa, con el objeto de que cualquier persona ajena a él (o el mismo autor) pueda entender qué hace y cómo lo hace.

a) Análisis del problema
El primer paso para encontrar la solución a un problema mediante una computadora es analizarlo. Se requiere definir el problema de la manera más precisa posible.
Dado que se busca una solución, se debe examinar el problema cuidadosamente con el fin de identificar que tipo de información se necesita producir. A continuación, se debe observar los elementos de información dados en el problema que puedan ser útiles para obtener la información. Por último debe generarse un procedimiento para producir los resultados deseados, a partir de los datos, y que será el algoritmo.

b) Diseño del algoritmo
Durante el análisis del problema se determina que debe hacer el programa. Con el diseño se reconoce como el programa va a realizar la tarea solicitada.
La forma habitual de abordar la resolución de un problema complejo es dividirlo en sub-problemas y a continuación dividir estos en otros de nivel más bajo, hasta que pueda implementarse una solución en la computadora. Esta forma de diseñar algoritmos lo veremos en el capitulo 5.

El diseño del algoritmo es independiente del lenguaje de programación en el que se vaya a codificar posteriormente, aunque en la práctica suele ser necesario considerar también las características (ventajas y limitaciones) de los lenguajes "candidatos".

c) Codificación
Codificar es escribir, en lenguaje de programación, la representación del algoritmo desarrollado en las etapas precedentes. El resultado se denomina **"programa fuente".**

d) Compilación y ejecución
Una vez que el algoritmo se convirtió en "programa fuente" mediante el proceso de codificación, es preciso traducirlo a código o lenguaje de máquina, el único que la computadora es capaz de entender y ejecutar. El encargado de realizar esta función es un programa traductor (compilador o intérprete). Si luego de la compilación se presentan errores (conocidos como **"errores de compilación o sintácticos"**) es preciso, entonces, modificar el código del programa de forma que éste se adapte a las reglas de sintaxis del lenguaje elegido, con el fin de corregir los errores.

Este proceso continúa hasta que ya no se producen errores sintácticos, con lo cual el compilador genera un "programa objeto", aún no ejecutable. Una vez ejecutado, y suponiendo que no hay errores durante su ejecución denominados **"errores de ejecución o semánticos",** se obtendrá la salida de los resultados del programa.

e) Verificación y depuración
La verificación y la depuración son procesos sucesivos mediante los que comprueba un programa con una amplia variedad de datos de entrada ("datos de prueba"), para determinar si hay errores, identificarlos y corregirlos. Para realizar la verificación y la depuración se debe desarrollar una amplia gama de datos de prueba: valores normales de entrada, valores extremos, valores con salida o resultado conocido y que comprueban aspectos esenciales del programa.

Los errores más difíciles de detectar y, por lo tanto, los más peligrosos, son los **errores semánticos**, debido a un mal diseño del algoritmo. No saltan a la vista como errores de compilación ni de ejecución y solo pueden advertirse por la obtención de resultados incorrectos -cuando se conoce el resultado exacto de los cálculos o se tiene idea del orden o magnitud de ellos- por lo que la importancia de esta fase resulta crucial. Son errores invisibles en un programa que no puede detectar el compilador, ya que no son el resultado de un uso incorrecto del lenguaje de programación, sino de decisiones erróneas durante el proceso de análisis del problema o de la mala formulación de algoritmos.
Una vez detectados estos errores hay que volver a rediseñar el algortimo, codificar y compilarlo, para obtener el código ejecutable correcto.

A continuación veremos algunos ejemplos de los errores en un programa Pascal:

Errores sintácticos: son los que se cometen cuando no se siguen exactamente las reglas de escritura de un lenguaje de programación.
Ejemplos:

Program ConErrores;
Var
a,b,c: integer;
Begin
Writeln(' escriba el valor de a); [1]
Readln(a) [2]
Writeln(' escriba el valor de b');
Readln(b);
c=a+b; [3]
Writeln(' el resultado es ') c ; [4]
Readln;
End.
	Program SinErrores;
	Var
	a,b,c: integer;
	Begin
	Writeln(' escriba el valor de a');
	Readln(a);
	Writeln(' escriba el valor de b');
	Readln(b);
	c:=a+b;
	Writeln(' el resultado es ', c) ;
	Readln;
	End.

[1] No cierra la comilla de la frase a escribir [2] No finaliza la sentencia con ; [3] Faltan los : en la asignación [4] La variable c debe estar en el cuerpo entre paréntesis la sentencia writeln	

Errores semánticos: son los que se cometen cuando los resultados obtenidos no son los esperados como solución al problema planteado.
Ejemplo: el objetivo del siguiente programa es calcular el promedio entre dos números

Program ConErrores;
Var
a,b,c: real;
Begin
Writeln(' escriba el valor de a');
Readln(a);
Writeln(' escriba el valor de b');
Readln(b);
c:=a+b/2; [1]
Writeln(' el promedio es ', c) ;
Readln;
End.
	Program SinErrores;
	Var
	a,b,c: real;
	Begin
	Writeln(' escriba el valor de a');
	Readln(a);
	Writeln(' escriba el valor de b');
	Readln(b);
	c:=(a+b)/2;
	Writeln(' el promedio es ', c) ;
	Readln;
	End.

[1] Al no escribir paréntesis, sólo b es dividido por 2 y el programa da un resultado que No es el promedio esperado	

e) Documentación
La documentación de un programa se clasifica en interna y externa.
La documentación interna se incluye en el código del programa mediante comentarios que ayudan a comprenderlo. Para funcionar un programa no necesita de comentarios. Su finalidad es hacer que los programas sean más fáciles de leer y comprender.
La importancia de una documentación interna es todavía mayor cuando los programas son complejos, tienen muchas líneas y es difícil su seguimiento (lectura e interpretación).

En general, la documentación externa se compone de un manual de instalación, un manual del usuario y un manual de mantenimiento.

b) Razonamiento top-down (o deducción de arriba hacia abajo, o diseño descendente)

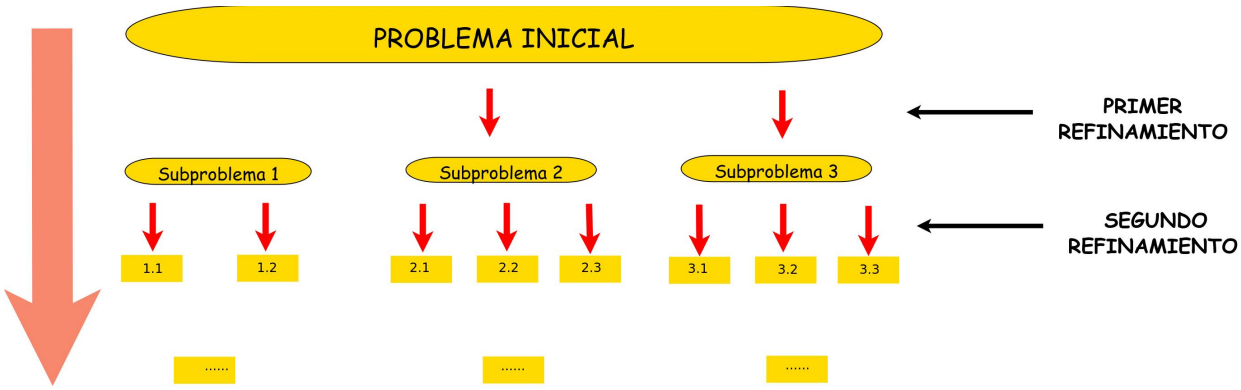
La técnica de diseño descendente (en inglés, top-down) está basada en un proceso de aproximación sucesiva a la solución del problema planteado. El apelativo de diseño descendente surge del hecho de que se parte de una especificación abstracta del problema por resolver para, mediante refinamientos sucesivos, ir "descendiendo" hasta cubrir todos los detalles y describir el programa en un lenguaje de programación.

También existe la técnica contraria, llamada diseño ascendente (en inglés, bottom-up), que parte de soluciones concretas disponibles para diferentes partes del problema y las integra para generar la solución total.

En cierto modo, la técnica de refinamiento progresivo se puede comparar con la actitud de un escultor ante un bloque de mármol con el objetivo de obtener una figura humana: para empezar, usando el cincel y el martillo grandes, proceder a esbozar sin mucho miramiento una figura humanoide con cabeza, tronco y extremidades; posteriormente, y ya con útiles de precisión, comenzará la labor de refinamiento y la obtención de detalles específicos.

En el diseño descendente, se parte de una especificación en lenguaje natural del problema que se quiere resolver y, sucesivamente, se va "depurando" poco a poco, perfilándose mejor las distintas partes del programa, apareciendo unos detalles acabados y otros a medio camino. El proceso de refinamiento continúa hasta que finalmente se obtiene una versión en la que el nivel de detalle de los objetos y las acciones descritos puede expresarse de forma comprensible para la computadora.

El diseño descendente de programas consiste, entonces, en identificar las funciones a cumplir por el programa al nivel mayor (global) y luego proceder desde ellas hacia una identificación de funciones menores que deriven de las anteriores hasta niveles en que la potencia del software pueda resolverlas. Esto es, se continúa el proceso de división abstracta en piezas, cada vez más elementales, hasta llegar a un nivel lo suficientemente simple para ser realizado fácilmente por una máquina real. Resumiendo, la deducción de arriba hacia abajo implica ir de lo abstracto (acciones no primitivas) a lo particular (acciones primitivas).



Ejemplo:
Cálculo del seno usando un polinomio de Taylor

$$\mathrm{Sen}(x) \simeq p_7(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!}$$

Vamos a proponer un algoritmo, iniciando un desarrollo top-down, con una acción no primitiva:

ALGORITMO SENO1

1. Leer x
2. calcular $p_7(x)$ {acción no primitiva}
3. Escribir el resultado

Veamos este algoritmo más refinado:

ALGORITMO SENO2

1. Leer x

2.1 Calcular los términos de la serie
2.2 Sumar algebraicamente los términos de la serie
3. Escribir el resultado

Vamos a descomponer el paso 2 en acciones más elementales:

ALGORITMO SENO3

1. Leer x

2.1.1 $a \leftarrow x^3/3!$
2.1.2 $b \leftarrow x^5/5!$
2.1.3 $c \leftarrow x^7/7!$
2.2 sen ← x - a + b-c
3 Escribir sen

Y, si suponemos que nuestro procesador no sabe calcular factoriales ni potencias, pero sí sabe multiplicar, podemos refinar el algoritmo así:

ALGORITMO SENO4

1. Leer x

2.1.1.1 pot1 ← x*x*x
2.1.1.2 fact1 ← 3*2
2.1.1.3 a ← pot1/fact1
2.1.2.1 pot2 ← x*x*pot1
2.1.2.2 fact2 ← 5*4*fact1
2.1.2.3 b←pot2/fact2
2.1.3.1 pot3 ←x*x*pot2
2.1.3.2 fact3 ←7*6*fact2
2.1.3.3 c←pot3/fact3
2.2 sen ← x - a + b-c
3 Escribir sen

Como se ha visto, el razonamiento top-down ayuda a analizar nuestro problema, partiendo del enunciado (la acción no primitiva más importante) para luego refinar hacia acciones más elementales, hasta alcanzar las acciones primitivas de nuestro procesador.

Para lograr programas correctos mediante el método de los refinamientos sucesivos, debe asegurarse la corrección en cada paso del desarrollo.

Para problemas de gran envergadura, la metodología de diseño descendente facilita la tarea de programación en equipo, puesto que en las distintas etapas de refinamiento se pueden emplear distintas personas para que se dediquen al refinamiento de distintos bloques del programa.

2. Datos subrango

Un tipo de dato subrango puede definirse como un subintervalo de los tipos ordinales asociados. La definición de un tipo por subrango indica simplemente el valor menor y mayor en el subrango, teniendo en cuenta que la cota inferior no puede ser mayor que la cota superior. *No se permiten* subrangos del tipo real**.** Las variables de este tipo poseen todas las propiedades de los tipos asociados.

Sintaxis:

TYPE
Identificador de tipo=constante1..constante2;

Ejemplo:

```
TYPE
 Siglo_XXI=2001..3000; {subrango de integer}
 Minusculas='a'..'z'; {subrango de char}

VAR
 D1,D2,D3:Siglo_XXI;
 letra:Minusculas;
```

3. Expresiones

La combinación de constantes y variables mediante ***operadores*** generan expresiones de los siguientes tipos:

- *Aritméticas*
- *Relacionales*
- *Lógicas*
- *Carácter o cadena*

La característica fundamental de una expresión es la obtención de ***un valor*** luego de su cálculo. El formato de las expresiones puede ser:

- ***operando*** **operador** ***operando*** (expresión binaria). Por ejemplo a+b
- **operador** ***operando*** (expresión unaria). Por ejemplo -b

Los operandos pueden ser constantes, variables o mediante una definición recursiva, los operandos pueden ser expresiones.

3.1 Expresiones aritméticas

Los siguientes operadores aritméticos generan valores enteros cuando son aplicados a operandos enteros:

*	Multiplica
Div	Divide y trunca
Mod	a mod b= a-[(a div b)*b]
+	Suma
-	Resta

Observación: los operadores Div y Mod SÓLO pueden aplicarse a operandos enteros. Por ejemplo, es ilegal la operación 7 div 2.0

Ejemplos

```
7 div 2      (su resultado es 3)
17 mod 3     (su resultado es 2)
```

En los siguientes operadores aritméticos, si al menos uno de los operandos es de tipo real (el otro podría ser de tipo entero) **se produce un valor real**:

*	Multiplica
/	División (ambos operandos pueden ser enteros, pero el resultado es siempre real)
+	Suma
-	Resta

Ejemplos

```
7/2 (su resultado es 3.5) ¿por qué sólo se obtiene este resultado?
12/6.2
(7 div 2)+(12/6.2)
(7 div 2)+(12/6.2) - A*2.1
beta * alfa / 16.87654
```

3.2 Expresiones relacionales

Los operadores relacionales sirven para comparar dos valores por mayor, menor, igual, mayor o igual, menor o igual, o distinto. El resultado es un valor booleano (true o false).

=	Igual
<>	Distinto
>	mayor
<	menor
<=	Menor o igual
>=	Mayor o igual

Ejemplos:

4.5>22.3 resultará ***false***
2.2=2.2 resultará ***true***
A<=2.3 el resultado depende del valor de A
(w+7.8)=(a/b-9) el resultado depende del valor de w, a y b.

3.3 Expresiones lógicas

Las expresiones con valor booleano (las relacionales) pueden combinarse mediante los operadores lógicos: and (conjunción), or (disyunción) y not (negación).

Si p y q son operandos lógicos tenemos las siguientes tablas de verdad:

P	q	p and q	p or q	not p
True	true	true	true	false
True	false	false	true	false
False	true	false	true	true
False	false	false	false	true

Ejemplos:

```
(7>2) and (55=5)
(Altura>=2) or ((peso+2=c) and (z/(5*c) >0))
```

Observaciones:

- Se considera como expresiones (las más elementales) a una constante y a una variable.
- Cuando se arman expresiones se debe tener en cuenta las siguientes reglas de precedencia a fin de colocar correctamente los paréntesis si se requiere alterar dicha prioridad:

Operador	Prioridad
Not	1 (la mas alta, se evalúa primero)
*, /, div, mod, and	2
+,-,or	3
<,<=,=,<>,>=,<	4 (la mas baja, se evalúa al final)

3.4 Expresiones cadenas

El operador usado es "+" y sirve para concatenar (unir) caracteres y/o cadenas de caracteres.

Ejemplo:

'o'+'s'+'o'	quedará	'oso'
'oso'+' '+'blanco'	quedará	'oso blanco'

Observación: En Pascal existen funciones predefinidas (listas para ser usadas) y pueden formar partes de las expresiones.

3.5. Ejemplo integrado

Calcular el promedio de las tres notas de Computación del alumno Alejandro Martinez.

ALGORITMO PROMEDIO

1) Ingresar nombre del alumno
2) Ingresar las tres notas
3) Calcular el promedio
4) Imprimir el resultado y nombre

Programa PASCAL:

```
Program promedio;

VAR
      nombre = string[25];
      nota1,nota2,nota3 : integer;
      promedio : real;
      ap_y_nom : nombre;
BEGIN
      Write('Ingrese apellido y nombre del alumno');
      Readln(ap_y_nom);
      Write('Ingrese las tres notas de computación');
      Readln(nota1,nota2,nota3);
      promedio:=(nota1+nota2+nota3)/3;
      Writeln(ap_y_nom,' tiene como promedio ',promedio)
END.
```

4. Funciones Predefinidas

El lenguaje Pascal incluye un número determinado de funciones estándar predefinidas que se encuentran ya incorporadas y no necesitan ser declaradas y codificadas en el programa. Se hallan disponibles funciones matemáticas, funciones de cadenas y otras específicas que permiten resolver problemas numéricos, de simulación, estadísticos, números de bytes ocupados por los argumentos, etc. A continuación se describen las funciones predefinidas más utilizadas.

4.1 - Funciones aritméticas y trigonométricas.

Nombre	Tipo del argumento	Resultado	Tipo del resultado
Abs(x)	Real o Integer	el valor absoluto del argumento	Real o Integer
ArcTan(x)	Real o Integer	el arco tangente del argumento	Real
Cos(x)	Real o Integer	el coseno del argumento	Real
Exp(x)	Real o Integer	el exponencial del argumento	Real
Frac(x)	Real	la parte decimal del argumento	Real
Int(x)	Real	la parte entera del argumento	Real
Ln(x)	Real o Integer	el logaritmo natural del argumento	Real
Round(x)	Real o Integer	el entero más próximo al argumento	Integer
Sin(x)	Real o Integer	el seno del argumento	Real
Sqr(x)	Real o Integer	el cuadrado del argumento	Real
Sqrt(x)	Real o Integer	la raíz cuadrada del argumento	Real
Trunc(x)	Real	la parte entera del argumento	Integer
Pi	no posee	3.1415926535897932385	Real

Nota 1: El argumento de las funciones trigonométricas está expresado en radianes.
Nota 2: No existe operador exponencial en Pascal. Para calcular x a la i-ésima potencia, se emplea : exp(i*ln(x))

4.2 - Funciones ordinales

Función	Resultado
Ord(x)	devuelve el número ordinal correspondiente al argumento.
Chr(x)	devuelve el carácter en el código ASCII correspondiente al argumento
Succ(x)	devuelve el sucesor del argumento, produce error si no tiene sucesor
Pred(x)	devuelve el predecesor del argumento.

5. EJERCICIOS

1) En el siguiente programa se han cometido errores sintácticos. Copiarlo, compilarlo y escribir los mensajes de error que muestra Pascal durante la compilación. Observar, en cada caso, la posición en que se detiene el cursor.

Programa	Mensajes de Error del Compilador

`Program errores;`
`  var`
`  p,a,c:integer;`
 	
`begin`
`  Writeln('Escriba el valor a');`
`  Readln(a)`
`  c=10;`
`  b:=a+4;`
`  writeln('El valor de b es ',b);`
`  p:=a/c;`
`  writeln('El resultado es ',p);`
`end.`	

2) Descubrir los errores de los siguientes programas Pascal. Corregirlos indicando cuáles son sintácticos y cuáles semánticos, y finalmente ejecutar el programa corregido. Agregar carteles aclaratorios para facilitar el ingreso de datos.

a) Objetivo: Calcular el promedio de tres números enteros.

```
Program promed;
var
      p,a,b,c:integer;
begin
      readln(a,b,c);
      p:=a+b+c/3;
      writeln('El promedio es ',p);
end.
```

b) Objetivo: Determinar el perímetro de una circunferencia conociendo la longitud del radio.

```
Program longi;
var
  r:real;
  L:integer;
Begin
  readln(r);
```

```
  L:=PI*r*r;
  writeln("La longitud es ", L);
end.
```

c) Objetivo: Calcular las raíces reales de una ecuación de segundo grado: $Ax^2+Bx+C = 0$

```
programa RAIZ;
const
  A=2;
  B=10;
var
  C:real; X1,X2:integer;
begin
  writeln("Cálculo de raíces de la ecuación de 2do. grado");
  writeln;
  writeln("A=";A;" B=";B);
  write("Ingrese C:");
  readln(C);
  X1:=-B+sqrt(sqr(B)-4AC)/2A;
  X2:=-B-sqrt(sqr(B)-4AC)/2A;
  writeln("El valor de X1 es X1 y el de X2 es X2");
end.
```

3) Calcular la superficie de un triángulo en base a la medida de sus lados. Todos los valores mostrados por pantalla deben expresarse **con dos cifras decimales**. La superficie puede calcularse como:

$S = \sqrt{P*(P-LADO1)*(P-LADO2)*(P-LADO3)}$, dónde

$$P=\frac{LADO1+LADO2+LADO3}{2}$$

4) Se leen los 3 coeficientes de un polinomio P(x) de 2do grado, determinar el valor P(a) para un 'a' que se ingrese como dato por teclado.

5) El siguiente programa tiene el objetivo de calcular la potencia n-ésima de un número: X^n.

Considerar que $x^m = e^{m \cdot \ln(x)}$ y en Pascal existe la función predefinida exp(m) que permite hallar e^m

Editar y compilar el programa dado y probarlo para calcular 7^4, $64^{\frac{1}{3}}$, $4^{1.5}$y $(-7)^4$. Analizar qué sucede al ejecutar el programa en cada caso.

```
Program prueba;
     Var
       x,y:real;
       n: byte;
     Begin
       Writeln('Escriba el valor de la base x');
       Readln(x);
       Writeln('Escriba el valor del exponente n');
       Readln(n);
       y:=exp(n*ln(x));
       Writeln('El resultado es ', y:5:2);
       Readln;
     End.
```

6) Evaluar las siguientes funciones en un valor x que se ingresa por teclado:

a) $f(x) = \dfrac{4(1-x^2)^2}{(1+5x)^2}$

b) $f(x) = -6e^{-4x^2}$

c) $f(x) = \left(1+\dfrac{x}{n}\right)^n$, donde n es un número natural que se ingresa por teclado

Nota: determinar en cada caso las restricciones sobre x ó sobre n. Para calcular la potencia sólo utilizar la expresión del ejercicio anterior si no existe otra posibilidad.

7) Determinar el valor de la longitud de un cateto (B) de un triángulo rectángulo, conociendo el valor de la longitud del otro cateto y del ángulo agudo adyacente a C. El valor del ángulo se ingresa en sistema sexagesimal (en grados, minutos y segundos). Informar los resultados con 3 cifras decimales.

8) Calcular la solución de un sistema de dos ecuaciones lineales con dos incógnitas utilizando el método de determinantes (se supone compatible y determinado).

9) Se tiene el objetivo de intercambiar el contenido de dos variables cuyo valor se ingresa por teclado. Por ejemplo: si A=3 y B=2 luego A=2 y B=3. Se escribe el siguiente programa:

```
Program prueba;
     Var
       A,B:integer;
     Begin
       Writeln('Escriba el valor de A');
       Readln(A);
       Writeln('Escriba el valor de B');
       Readln(B);
       A:=B;
       B:=A;
       Writeln('Ahora A es ', A, ' y B es ', B);
       Readln;
     End.
```

a) Realizar la prueba de escritorio para el lote del ejemplo y justificar por qué no cumple su objetivo.
b) Modificar el programa anterior para que cumpla su objetivo.

10) Se tienen una cubeta con 400 cm^3 de disolvente y tres recipientes, de volúmenes v1, v2 y v3 respectivamente, que contienen ácidos. Se incorpora a la cubeta el 5% de cada uno de los tres ácidos. Luego se añade un estabilizador en un proporción que representa 1/3 (un tercio) del volumen final.
Calcular y escribir el volumen del estabilizador y el volumen final que contendrá la cubeta expresándolos con 2 decimales.

11) Ingresar un número entero positivo de 4 dígitos y mostrar su descomposición numérica en base decimal.

Ejemplo: X=1568 debe mostrar **1** 10^3 + **5** 10^2 + **6** 10^1 + **8** 10^0

Sugerencia: considerar

1568	1000		568	100	
568	**1**		68	**5**	

12) Investigar cuál es la utilidad de las Funciones Predefinidas (Ord y Chr)

a) Editar y ejecutar el siguiente programa e investigar con el mismo cuáles son los códigos ASCII correspondientes a las letras del alfabeto.

```
Program carac;
     Var
        Car:char;
        Num: byte;
     Begin
        Writeln('escriba un carácter ');
        Readln(Car);
        Writeln('El orden ASCII de este carácter es ', ORD(Car));
        Writeln('Escriba un número entero positivo');
        Readln(Num);
        Write('El carácter que corresponde al número ASCII dado es ',CHR(Num));
        Readln;
     End.
```

b) Escribir un programa que, en función de lo investigado en el inciso anterior, dada una letra mayúscula la convierta en minúscula

13) Mostrar la salida del siguiente programa con el lote de prueba dado:

Lote de prueba: mercurio, mano, 18

```
Program BOOLEANOS;
const
 a=5>6;
 d=3;
 b=odd(d+4);
 c=true;
var
 resultado, anterior, divisible:boolean;
 Pal1, Pal2:string[10];
 M:integer;
begin
 resultado:= c and not(a);
 writeln(resultado);
 resultado:=resultado or b;
 writeln(resultado);
 Writeln('Escriba dos palabras');
```

```
 Readln(Pal1);
 Readln(Pal2);
 anterior:= Pal1<Pal2;
 Writeln('Decir que ',Pal1,' es alfabéticamente anterior a ',Pal2,' es ',anterior);
 Writeln('Ingrese un valor entero');
 Readln(M);
 divisible:=(M mod 3)=0;
 Writeln('Decir que el número ingresado es divisible por 3 es ', divisible);
 readln;
end.
```

14) Para s=6, t=0 y u=20, determinar los valores de las variables m, n, o, p, q.

```
m := s < t;
n := not m  and (u<25);
o := m or ((s-t) mod 2 = 0);
p := m and (t>0) or o;
q := m and ((t>0) or o);
```

15) Dado el programa escrito más abajo:
a) Realizar la prueba de escritorio.
b) Editarlo, compilarlo y ejecutarlo.
c) Fundamentar los resultados obtenidos.

```
program almacena;
var
    a,b,c: real;
    h:boolean;
Begin
    a:=2;
    b:=sqrt(2); { sqrt calcula raíz cuadrada }
    c:=sqr(b);    { sqr eleva al cuadrado }
    h:=a=c;
    writeln('La igualdad es ', h);
    readln;
end.
```